

Session 6: Arrays (C Version)

Topics Covered 1. Array Theory 2. 1D and 2D arrays 3. Array Initialization 4. Accessing Array Elements 5. Passing Arrays to Functions 6. Common Array Problems 7. Activities

1. Array Theory

2. **Definition:** An array is a collection of elements of the same type stored in contiguous memory locations.

3. Types:

4. **1D Array:** Linear arrangement of elements.

5. **2D Array:** Matrix or table arrangement (rows × columns).

6. Advantages:

7. Easy to store and access multiple elements using indices.

8. Efficient memory utilization due to contiguous allocation.

9. Disadvantages:

10. Fixed size (cannot dynamically resize in C without dynamic memory allocation).

11. Insertion and deletion can be costly as elements may need shifting.

12. Terminology:

13. **Index:** Position of an element in the array (starting from 0).

14. **Length/Size:** Total number of elements in the array.

15. **Contiguous memory:** Elements are stored in sequential memory locations.

16. Applications:

17. Storing data like scores, temperatures, or large datasets.

18. Useful in implementing algorithms like sorting, searching, and dynamic programming.

19. 1D and 2D Arrays

20. 1D Array: stores elements linearly.

```
int arr[5];  
int arr2[] = {1, 2, 3, 4, 5};
```

21. 2D Array: stores elements in rows and columns.

```
int matrix[3][4];  
int matrix2[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
```

22. Array Initialization

23. 1D: `int arr[] = {1,2,3};`

24. 2D: `int mat[2][2] = {{1,2},{3,4}};`

25. Accessing Array Elements

26. Use indices starting from 0.

```
for(int i=0;i<5;i++) printf("%d\n", arr[i]);
for(int i=0;i<3;i++){
    for(int j=0;j<3;j++) printf("%d ", matrix2[i][j]);
    printf("\n");
}
```

27. Passing Arrays to Functions

28. Arrays are passed as pointers.

```
void updateArray(int arr[], int size){ arr[0] = 100; }
int nums[] = {1,2,3}; updateArray(nums,3);
printf("%d\n", nums[0]); // 100
```

29. Common Array Problems

30. Sequential Search

```
int sequentialSearch(int arr[], int size, int target){
    for(int i=0;i<size;i++) if(arr[i]==target) return i;
    return -1;
}
```

31. Binary Search (Iterative & Recursive)

```
int binarySearchIter(int arr[], int size, int target){
    int left=0,right=size-1;
    while(left<=right){
        int mid=left+(right-left)/2;
        if(arr[mid]==target) return mid;
        if(arr[mid]<target) left=mid+1; else right=mid-1;
    }
    return -1;
}
int binarySearchRec(int arr[], int left, int right, int target){
    if(left>right) return -1;
    int mid=left+(right-left)/2;
    if(arr[mid]==target) return mid;
    if(arr[mid]<target) return binarySearchRec(arr,mid+1,right,target);
}
```

```

    return binarySearchRec(arr, left, mid-1, target);
}

```

32. Remove Element

```

int removeElement(int arr[], int size, int val){
    int k=0;
    for(int i=0;i<size;i++) if(arr[i]!=val) arr[k++]=arr[i];
    return k;
}

```

33. Remove Duplicates from Sorted Array

```

int removeDuplicates(int arr[], int size){
    if(size==0) return 0;
    int k=1;
    for(int i=1;i<size;i++) if(arr[i]!=arr[i-1]) arr[k++]=arr[i];
    return k;
}

```

34. Plus One

```

#include <stdlib.h>
int* plusOne(int* digits, int size, int* returnSize){
    for(int i=size-1;i>=0;i--){
        if(digits[i]<9){ digits[i]++; *returnSize=size; return digits; }
        digits[i]=0;
    }
    int* res=(int*)malloc((size+1)*sizeof(int));
    res[0]=1; for(int i=1;i<=size;i++) res[i]=0;
    *returnSize=size+1; return res;
}

```

35. Single Number

```

int singleNumber(int arr[], int size){
    int res=0;
    for(int i=0;i<size;i++) res^=arr[i];
    return res;
}

```

36. Contains Duplicate

```

#include <stdbool.h>
#include <stdlib.h>
bool containsDuplicate(int arr[], int size){
    for(int i=0;i<size;i++){
        for(int j=i+1;j<size;j++) if(arr[i]==arr[j]) return true;
    }
    return false;
}

```

37. Rotate Array

```

void reverse(int arr[], int start, int end){
    while(start<end){ int temp=arr[start]; arr[start]=arr[end];
arr[end]=temp; start++; end--; }
}
void rotate(int arr[], int size, int k){
    k%=size;
    reverse(arr,0,size-1);
    reverse(arr,0,k-1);
    reverse(arr,k,size-1);
}

```

38. Intersection of Two Arrays

```

#include <stdlib.h>
int* intersection(int* nums1, int size1, int* nums2, int size2, int*
returnSize){
    int* res=(int*)malloc(sizeof(int)*size1);
    int k=0;
    for(int i=0;i<size1;i++){
        for(int j=0;j<size2;j++){
            if(nums1[i]==nums2[j]){ res[k++]=nums1[i]; break; }
        }
    }
    *returnSize=k; return res;
}

```

39. Activities **Program: Find Largest Element**

```

#include <stdio.h>
int main(){
    int arr[]={10,25,5,40,15};
    int max=arr[0];
}

```

```
for(int i=1;i<5;i++) if(arr[i]>max) max=arr[i];  
printf("Largest element is: %d\n", max);  
return 0;  
}
```